

CREATUSH

TweenEffects Pro

Modular DOTween Animation System for Unity
Technical Documentation | v2.0

Version	2.0.0
Unity	2022.3 LTS or later
Dependency	DOTween (free or Pro)
Namespace	Creatush.TweenEffectsPro
Assembly	Creatush.TweenEffectsPro.Runtime

1. Overview

TweenEffects Pro is a modular, data-driven animation system for Unity built on DOTween. It replaces the Animator for UI and gameplay animations — no state machines, no clips, no overhead. Every animation is composed at runtime from plain C# data, played through a single controller.

Architecture

- **EffectDefinition** — abstract base class. Plain **[Serializable]** C# data, not a MonoBehaviour. Holds timing, ease, and shared helpers.
- **Effects** — concrete EffectDefinition subclasses. Each does one focused thing.
- **MasterSequenceController** — the single controller. Holds an ordered list of EffectSteps and composes them into one DOTween Sequence.
- **EffectStep** — one timeline entry: which effect (inline or a shared preset), which target(s), and how it's timed relative to the steps before it.
- **Support layer** — EffectCatalogue, EffectPresetSO, TweenSettingsSO, SplinePathSO.

Note: Version 2.0 replaces the previous SingleEffectController / SequenceEffectsController / ParallelEffectsController / StaggerEffectsController split with one MasterSequenceController. Sequential, joined (“parallel”), and staggered-across-targets are now just properties of a step, not separate component types. See Section 10 (Migration History) for how projects built on the pre-2.0 components were carried forward.

Assembly Structure

Assembly	Contents
...Runtime	All runtime code — effects, MasterSequenceController, base classes, SOs
...Editor	MasterSequenceControllerEditor, EffectPresetSOEditor, EffectTypeMenu, SplinePathEditor, property drawers

Note: Place the Runtime .asmdef at Runtime/ root. Place the Editor .asmdef at Editor/ root. Do not nest .asmdef files inside subfolders. Editor files (anything using UnityEditor) must live inside Editor/.

2. Setup

Requirements

- Unity 2022.3 LTS or later
- DOTween (free) or DOTween Pro — import before TweenEffects Pro

Folder Structure

```
Assets/Creatush/TweenEffectsPro/  
  Runtime/                    <- Creatush.TweenEffectsPro.Runtime.asmdef  
  Base/                      EffectDefinition, EffectContext, IEffectLifecycle,  
                             EaseCurves, Attributes  
  Effects/                   All EffectDefinition subclasses (+ SplinePath/)  
  Controllers/               MasterSequenceController, EffectStep  
  SOs/                       EffectCatalogue, EffectPresetSO, TweenSettingsSO  
  Editor/                    <- Creatush.TweenEffectsPro.Editor.asmdef  
  MasterSequenceControllerEditor.cs  
  EffectPresetSOEditor.cs  
  EffectTypeMenu.cs  
  SplinePathEditor.cs  
  CurvePresetDrawer.cs  
  CatalogueKeyDrawer.cs  
Assets/Resources/  
  TweenSettings.asset        <- must be named exactly TweenSettings
```

Global Settings (TweenSettingsSO)

Create via: Assets > Create > Creatush > TweenEffects Pro > Global Settings. Place in any Resources/ folder named exactly TweenSettings. Applied automatically at startup via TweenSettingsLoader — no scene setup needed.

Field	Default	Description
globalTimeScale	1.0	Multiplies DOTween.timeScale at startup.
useUnscaledTime	false	Run animations independent of Time.timeScale. Use for pause menus.
tweenersCapacity	200	DOTween tween pool size. Increase to avoid runtime pool expansion.
sequencesCapacity	50	DOTween sequence pool size.
safeMode	true	DOTween safe mode — catches tween errors instead of throwing.
logWarnings	true	Log setup warnings in the editor.

3. EffectDefinition — Base Class

Abstract base for every effect. Provides timing, ease, and leak prevention. Subclasses implement `BuildSequence(ctx)` and `GetDuration()`. Effects are plain data — not `MonoBehaviours` — and carry no scene identity of their own; `BuildSequence()` is always given an explicit `EffectContext` instead of reading its own transform or `gameObject`.

EffectContext

Everything an effect needs to build its Sequence for one play, passed into `BuildSequence()`:

Field	Description
target	The Transform this play of the effect acts on.
owner	The GameObject of the MasterSequenceController driving this effect. Used for SetLink/kill scoping and as a fallback parent for effects that spawn objects.
index / totalCount	Position and size of this target within a stagger/multi-target group. 0 / 1 for a single target.
textRenderer / characterIndex	Reserved for a future per-character TMP text effect. Unused by every effect today; ignored otherwise.

Abstract Members

Member	Description
BuildSequence(EffectContext ctx)	Build and return a DOTween Sequence. Always end with <code>return FinaliseSequence(seq, ctx.owner)</code> .
GetDuration()	Return wall-clock seconds for one play. Must not allocate. Used by MasterSequenceController to plan stagger/timeline timing.

Shared Fields

Field	Default	Description
duration	0.5s	Length of one play in seconds.
easeMode	Preset	Preset = DOTween Ease enum. Curve = custom AnimationCurve.
ease	OutQuad	Active when Ease Mode is Preset.
easeCurve	EaseInOut	Active when Ease Mode is Curve. [CurvePreset] attribute adds a picker.

Note: Looping is no longer configured per-effect. It's a whole-sequence concern set once on the owning MasterSequenceController (Auto Play > Loop), so a controller with several steps loops as one coherent unit instead of each effect repeating on its own, independently-timed cycle. See Section 6.

Protected Helpers

Method	Usage
ApplyEase(Tweener t)	Apply the configured ease. Returns the Tweener for chaining: seq.Append(ApplyEase(target.DOScale(...))).
FinaliseSequence(Sequence seq, GameObject owner)	Applies unscaled-time (if configured) and SetLink(owner, KillOnDisable). Always the last call before returning from BuildSequence().

Leak Prevention

FinaliseSequence() calls SetLink(ctx.owner, LinkBehaviour.KillOnDisable) — DOTween kills the sequence automatically when the owning GameObject is disabled. MasterSequenceController.Stop() adds a belt-and-suspenders DOTween.Kill(this) and per-target DOKill() for anything started outside the normal path.

Creating a Custom Effect

```
public class EffectMyEffect : EffectDefinition
{
    [SerializeField] private float myParam = 1f;
    public override float GetDuration() { return duration; }
    public override Sequence BuildSequence(EffectContext ctx)
    {
        if (ctx.target == null) return null;
        Sequence seq = DOTween.Sequence();
        seq.Append(ApplyEase(ctx.target.DOScale(myParam, duration)));
        return FinaliseSequence(seq, ctx.owner);
    }
}
```

4. Effects

All effects inherit `EffectDefinition` and support the shared ease and timing fields. Every effect calls `FinaliseSequence()` before returning.

EffectScalePop

Unified scale effect. Mode field selects the pattern.

Mode	Behaviour
ScaleIn	Animate from Start Scale to End Scale once.
Pulse	Animate to End Scale then back to Start Scale. Loops naturally when the controller loops.
Pop	ScaleIn with an elastic overshoot (Overshoot / Settle Duration). Springy entry feel.

EffectBounce

Position bounce with physically correct squash and stretch. Sequential phases: launch stretch → rise → apex unstretch → fall → landing squash → recovery spring. Squash Strength (0–1) controls deformation. Bounciness (0–2) controls the OutBack recovery overshoot.

EffectShake

Simultaneous shake on position, rotation, and/or scale — each independently toggleable. Rotation Strength is a `Vector3` for per-axis control (Z-only for a 2D screen shake). Vibrato controls oscillation count, Randomness controls directional spread, Fade Out smoothly decays the shake instead of cutting it.

EffectPunch

Elastic punch on any combination of position, rotation, and scale. All active axes run simultaneously via `seq.Join`. `ApplyEase` is not applied — DOTween's punch algorithm owns the decay curve. Shape the feel via Vibrato and Elasticity.

EffectMove

Local position from Start Position to End Position. Snaps to Start Position at play time so the prefab can be left in its idle state in the editor.

EffectMoveTransform

A-to-B move using two `RectTransform` references as waypoints. Drag any two `RectTransforms` into Origin and Destination. Automatically uses `anchoredPosition` for UI targets, `localPosition` otherwise.

EffectRotate

Local rotation from Start Rotation to End Rotation. Rotate Mode: Full360 = shortest path, FastBeyond360 = continuous spin (default), LocalAxisAdd = adds per loop.

EffectFloat

One rise-and-fall cycle per play; GetDuration() returns duration×2 (the full up-down cycle). For a continuously idling float, enable Loop on the owning MasterSequenceController — looping is a whole-sequence concern now, not something an effect opts into on its own. Ease defaults to InOutSine. Optional Tilt Degrees adds an organic lean.

EffectFade

CanvasGroup alpha from Start Alpha to End Alpha. Looks for a CanvasGroup on the target or its children; logs a warning and returns an empty sequence if none is found.

EffectColorTint

Graphic.color from Start Color to End Color. Works with any Unity UI Graphic (Image, Text, RawImage). Looks for the Graphic on the target or its children.

EffectFillAmount

Image.fillAmount from Start Fill to End Fill. Requires an Image set to Filled type — logs an actionable warning if it isn't.

EffectCrossfade

Crossfades a group of CanvasGroups out while a second group fades in, with a configurable Overlap — how far before the fade-out ends that the fade-in begins. duration drives the fade-out; Fade In Duration is separate.

EffectPivotOrbit

Orbits the target around a pivot offset from its own position. A temporary pivot GameObject is created at runtime for the duration of the effect and cleaned up automatically on complete or kill.

EffectSpriteSwap

Frame-by-frame sprite animation on a UI Image. Frames array; duration divided by frame count gives the per-frame interval. Supports reverse playback. No Animator required.

EffectSpriteAnimation

Full-featured sprite sheet playback on a UI Image, driven by its own frame rate rather than the shared duration field.

Field	Description
frames	Ordered sprite frames array.
frameRate	Frames per second.
playbackMode	Forward, Reverse, or PingPong.
speed	Playback speed multiplier (0.1–8). Also settable at runtime.
holdLastFrame	Keep the last frame displayed after a non-looping pass ends.
onAnimationComplete	Fired at the end of each complete pass through the frames.

EffectFollowSpline

Moves a target along a cubic Bézier spline defined by a SplinePath component. Features: arc-length remapping for constant speed, orient-to-path with 2D (Z-axis) and 3D (LookRotation) modes, scale-over-path AnimationCurve, roll interpolation between knots, and its own Loop/PingPong playback (splineLoopMode) independent of the controller's whole-sequence Loop. See Section 7.

Note: *EffectFollowSpline uses SetLink(KillOnDestroy) instead of KillOnDisable, so the sequence isn't killed if the source GameObject is disabled mid-animation.*

5. Effect Reward Fly

The classic “coins fly to the wallet” reward animation: spawns a burst of pooled items that pop in, float, then fly along a parabolic arc to a destination. Unlike most effects, EffectRewardFly manages its own pool of spawned GameObjects rather than animating a single target directly — the step's target is used only as the spawn source. It implements IEffectLifecycle to release its pooled instances when the owning MasterSequenceController is destroyed.

Setup

- Add a step to a MasterSequenceController and set its Effect to EffectRewardFly.
- Assign **Spawn Parent** — a RectTransform inside a Canvas that will parent the pooled items.
- Assign **Destination** — the RectTransform items fly toward (score counter, wallet icon, etc.).
- Assign **Item Prefab** — a UI Image with your coin/gem sprite.
- Call Play() on the controller from a button, sequence callback, or UnityEvent.

Fields

Field	Description
itemPrefab	Pooled item prefab. UI Image recommended.
itemCount	Pool size and number of items per play.
spawnParent	RectTransform parent for pool items. Must be inside a Canvas.
spriteFrames / spriteFrameRate	Optional sprite sheet animation while the item is visible.
spawnRadius / spawnRandomness	Scatter radius and cluster tightness around the source.
spawnStagger	Seconds between successive item spawns.
popInOnSpawn / popInDuration	Scale items from 0 on spawn with an OutBack ease.
floatDuration / floatHeight / floatSpeed	Sine-wave idle oscillation before flying.
destination	RectTransform items fly toward.
flyDuration / flyEase	Fly animation duration and ease.
arcHeight / arcVariance	Parabolic arc peak height and per-item random variation.
flyArrivalScale	Scale on arrival. 0 = shrink to nothing.
flyStagger	Delay between items starting their fly.

Field	Description
onAllArrived / onItemArrived	Events fired on each arrival and when all items have arrived.

Scene View Gizmo

MasterSequenceControllerEditor draws a live preview in the Scene View for any step resolving to EffectRewardFly, using Handles.matrix set to the spawn parent's local coordinate system. Shows: a spawn-radius disc, a destination dot, a float-range line, and an arc preview (solid = mean, faded = mean $\pm$ variance). All gizmo drawing is wrapped in try/finally so Handles.matrix is always restored.

6. MasterSequenceController

The single controller for TweenEffects Pro. Holds an ordered list of EffectSteps and composes them into one DOTween Sequence — sequential, joined (“overlapping”), and staggered-across-targets are all just properties of a step, not separate component types.

Public API

Method	Description
Play()	Build and play all steps forward.
PlayReverse(speed = 1)	Play the whole timeline reversed at the given speed multiplier.
PlayByTrigger(name)	Plays if name matches Triggers.playTrigger.
KillByTrigger(name)	Stops (including an active loop) if name matches Triggers.killTrigger.
Stop()	Kills all running tweens owned by this controller.
GetDuration()	Wall-clock duration of one Play(). Uses GetDuration() on each step's effect — no allocation.
AddStep(EffectStep)	Appends a step at runtime or from custom editor tooling.

Note: There is no separate Hide()/out-animation concept in v2.0. Play an exit animation the same way as an entry: either call PlayReverse() on the same controller, or configure a second MasterSequenceController for the exit and trigger it explicitly.

Auto Play & Loop

Field	Description
onStart	Play once on Start() — first activation only. Skipped if onEnable is also on.
onEnable	Play every time this GameObject is enabled.
loop	Loop this controller's sequence — baked into the sequence itself, so it applies no matter how playback was triggered (Auto Play, a script call, or a Trigger).
loopCount	-1 = infinite.
loopInterval	Seconds between the end of one run and the start of the next.
loopType	Restart: replay from the beginning each time. Yoyo: alternate forward/backward each loop, ping-ponging the whole sequence. Incremental: each loop continues from where the previous one ended, stacking rather than resetting.

Triggers

playTrigger / killTrigger are optional string identifiers external systems can use to Play() or Stop() this controller by name — wire PlayByTrigger()/KillByTrigger() to an Animation Event, a dialogue system, or any shared trigger bus. onSequenceComplete (UnityEvent) fires once the sequence finishes playing, after every loop iteration if looping; it never fires when Loop Count is -1 (infinite).

EffectStep

One entry in the controller's timeline. There is no separate “join previous” flag — a negative Delay overlaps with whatever came before, which is all “joining” ever needed to mean.

Field	Description
delay	Seconds from the end of the timeline so far. 0 = starts right after. Positive = adds a gap. Negative = overlaps/joins earlier steps.
preset	Optional EffectPresetSO. When assigned, its effect is used instead of the inline one below — lets the same configured effect be reused across controllers/prefabs.
effect	Inline [SerializeReference] EffectDefinition, used when no preset is assigned.
targetMode	Single: one target (Target Override, or this GameObject if empty). Multiple: an explicit target list, staggered. Children: every direct child of Target Override, staggered.
targetOverride	Single mode: the target. Children mode: the parent whose children are used. Ignored by Multiple mode.
targets	Explicit target list — Multiple mode only.
staggerInterval	Seconds between each target's start — Multiple/Children modes only.
reverseOrder / randomOrder	Reverse or shuffle (Fisher–Yates) target playback order.

Timeline Preview

The Inspector's Timeline Preview panel visualises every step as a bar along a shared track: a single blue-hue family distinguishes sequential steps, overlapping (negative delay) steps, and staggered (Multiple/Children target mode) steps by tone rather than by clashing colors, with a neutral grey band showing any delay before a bar starts. Bar width is proportional to that step's wall-clock duration, computed the same way GetDuration() computes it — no allocation, no need to enter Play Mode.

7. Spline Path System

SplinePathSO

Pure data and math asset. Cubic Bézier knots with position, tangentOut, tangentIn, and roll. Binary-search arc-length table for constant-speed remapping. All fields are HideInInspector — edited via SplinePathEditor's Scene View instead.

```
// Create via: Assets > Create > Creatush > TweenEffects Pro > Spline Path
```

SplinePath

Scene component. Holds a reference to a SplinePathSO asset and forwards all sampling calls to it — multiple GameObjects can share one asset. If no asset is assigned, a local inline instance is created automatically.

SplinePathEditor

Scene View editor for SplinePath. Features: knot handles (click to select), a full PositionHandle on the selected knot, tangent handles with dotted connecting lines, an Add Knot button, a Smooth Curves button (Catmull–Rom), a per-knot foldout in the Inspector, and playback override settings. All edits are Undo-wrapped.

8. Ease System

Every `EffectDefinition` supports two ease modes, selectable per effect. `EaseMode.Preset` uses DOTween's `Ease` enum. `EaseMode.Curve` uses a custom `AnimationCurve` with full tangent control.

EaseCurves Library

14 hand-tuned `AnimationCurve` presets. The `[CurvePreset]` attribute adds a preset picker and Apply button to any `AnimationCurve` field in the Inspector.

Group	Presets
Standard	Smooth Step, Ease Out, Ease In, Linear
Overshoot	Back Out, Back In, Back In Out
Bounce	Bounce Out, Bounce In
Elastic	Elastic Out, Elastic In
Specialty	Snap, Spring, Anticipate

9. Scriptable Objects

EffectPresetSO

Wraps a single configured EffectDefinition as a shareable asset, so the same effect can be reused across multiple MasterSequenceController steps and prefabs instead of re-authoring it inline every time. Assign one to an EffectStep's Preset field to have that step use it in place of its inline effect. Its custom editor draws the same effect-type picker used when adding a step to a controller.

```
// Create via: Assets > Create > Creatush > TweenEffects Pro > Effect Preset
```

EffectCatalogue

A named library of EffectPresetSO entries. Avoids duplicating configured effects across prefabs — define “CoinPop”, “DamageFlash”, “MenuSlideIn” once and reference by key. Lazy-builds a lookup dictionary on first Get() call; OnValidate invalidates the cache in the editor.

```
[SerializeField] private EffectCatalogue catalogue;  
[CatalogueKey(nameof(catalogue))]  
[SerializeField] private string effectKey;  
  
// Runtime:  
catalogue.Play("CoinPop", targetTransform);
```

TweenSettingsSO

Global project settings, covered in Section 2. Applied by TweenSettingsLoader at BeforeSceneLoad — no GameObject required. Must be in a Resources/ folder and named exactly TweenSettings.

10. Migration History

Version 2.0 replaced the pre-2.0 component split (SingleEffectController, SequenceEffectsController, ParallelEffectsController, StaggerEffectsController, and the MonoBehaviour-based VFXBehaviour effects) with the single MasterSequenceController + EffectStep model described in this document. Existing scenes and prefabs did not need a manual rewrite: the old components were temporarily kept under a Legacy namespace, and a MigrationTool converted them into the new model automatically, field-for-field, by reflection — every new EffectDefinition kept the exact same field names/types as its legacy counterpart specifically to make that possible.

Note: *That migration is now complete for this project. The pre-2.0 Legacy components and the MigrationTool have been removed — there's nothing left in the package to convert from. This section is kept as a historical record of how the upgrade worked, in case a future version needs the same playbook again.*

11. Quick Reference

Effect Selection

Goal	Effect
Scale entry	EffectScalePop (ScaleIn mode)
Idle scale pulse	EffectScalePop (Pulse mode, controller loop = true)
Springy pop	EffectScalePop (Pop mode)
Impact feedback	EffectPunch (punchScale = true)
Shake / damage	EffectShake
Physical bounce	EffectBounce
Panel slide	EffectMove or EffectMoveTransform
Idle float	EffectFloat (controller loop = true)
Continuous spin	EffectRotate (FastBeyond360, controller loop = true)
Fade in/out	EffectFade
Color flash	EffectColorTint
Crossfade panels	EffectCrossfade
Health bar	EffectFillAmount
Sprite animation	EffectSpriteAnimation
Sprite swap	EffectSpriteSwap
Path movement	EffectFollowSpline
Orbit	EffectPivotOrbit
Reward collection	EffectRewardFly

Step Configuration

There's one controller now, so "which controller" becomes "how is the step configured":

Goal	Configuration
One effect, one object	A single step, targetMode = Single.
Chain effects	Multiple steps in order, delay = 0 on each (or a positive gap).

Goal	Configuration
Overlap effects	A negative delay on the later step.
Same effect on many targets	targetMode = Multiple (explicit list) or Children, with staggerInterval.
Reusable configured effect	Assign an EffectPresetSO to the step's Preset field.
Exit animation	PlayReverse() on the same controller, or a second controller.